

# **LAB 3**

## **MULTI-AGENT CHATBOT WITH LANGGRAPH & GRADIO**

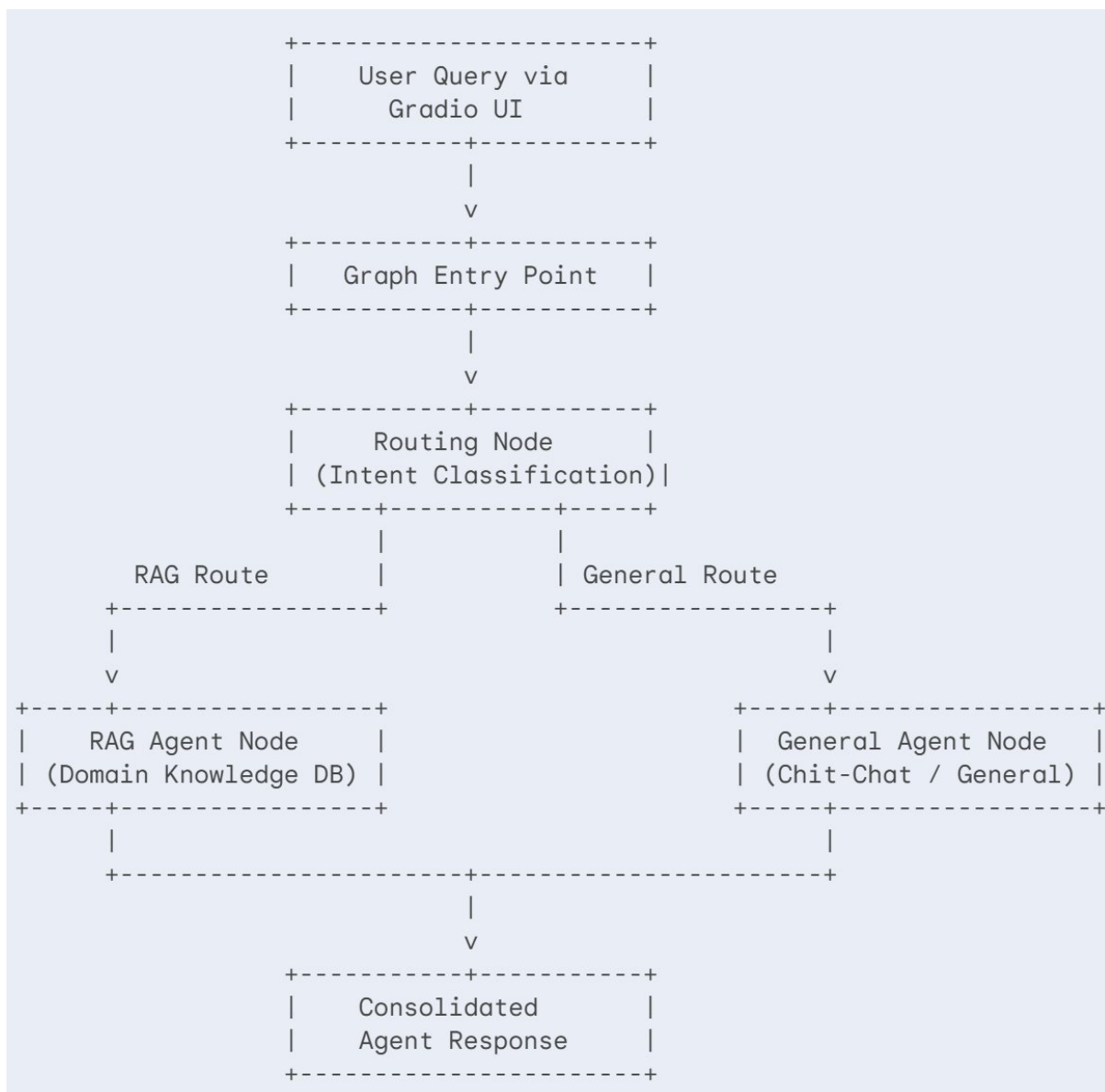
|               |                                 |
|---------------|---------------------------------|
| Course        | DDM501                          |
| Weight        | 5%                              |
| Format        | Team Lab (3-4 members per team) |
| Prerequisites | Lab 1 and Lab 2 completed       |

# 1. OVERVIEW

## 1.1. Introduction

The purpose of this lab is to build an advanced, state-aware conversational AI application: a Multi-Agent Chatbot using **LangGraph**, **LangChain**, and **Gradio**.

Unlike simple sequential linear chains (where LLMs process input in a fixed, single direction), real-world agentic workflows often require branching, looping, self-correction, and collaborative specialist behaviors. In this lab, you will design and construct a production-ready chatbot system capable of identifying user intent, dynamically routing traffic between specialized modules (a General Chatbot Agent and a Retrieval-Augmented Generation / RAG Agent), and presenting a clean, unified, real-time user interface.



## 1.2. Scenario: Enterprise Knowledge & Support Assistant

You have to build the Customer Support and internal Operations teams are currently using disjointed systems: one simple chat assistant for basic pleasantries/general knowledge, and another complex portal for querying internal technical manuals.

Your management team wants a single, unified chat interface.

Your task is to:

1. **Design a routing graph** using LangGraph to analyze user input.
2. **Implement a General Assistant Agent** for conversational interactions (Chit-Chat, general logic).
3. **Implement a RAG Assistant Agent** connected to a local vector store containing company-specific policy documents.
4. **Develop an LLM-based Routing Node** that reads incoming queries and seamlessly routes them to the correct agent node.
5. **Wrap the entire LangGraph workflow in an interactive Gradio UI** to provide an enterprise-grade web interface.

## 2. BACKGROUND

### 2.1. LangGraph & State Management

**LangGraph** is a library designed to build stateful, multi-actor applications with LLMs. Unlike standard agent models, LangGraph models agent behaviors as a **Graph** structure containing:

- **State:** A shared, structured data dictionary or object that is passed down to all nodes. Each node can read from or write updates to this central state.
- **Nodes:** Independent Python functions that receive the current State, perform computational tasks (like calling an LLM, querying a database, or invoking a tool), and return updates to modify the State.
- **Edges:** Mathematical connections between nodes that dictate execution flow. **Conditional Edges** execute routing functions to dynamically decide which node to transition to next based on the current contents of the state.

### 2.2. Multi-Agent Routing Architectures

Routing is a cornerstone pattern of Agentic AI. Instead of using a single giant system prompt to handle every edge case (which increases latency, prompt token overhead, and reduces accuracy), a **Router** categorizes incoming queries first.

- **Rule-Based Routers:** Use keyword matching or simple regex. These are highly performant but brittle.
- **Semantic / LLM Routers:** Prompt a small, fast LLM model to return a structured output (e.g., JSON or structured schema) classifying the query category. This provides superior semantic intelligence and maintains robustness across varying phrasings.

### 2.3. Retrieval-Augmented Generation (RAG)

RAG optimizes LLM outputs by querying an authoritative, external knowledge base prior to generating a response.

A classic RAG pipeline involves:

1. **Ingestion:** Splitting unstructured PDF/Markdown text into chunks.
2. **Vectorization:** Generating high-dimensional vector representations (embeddings) of text chunks

using model encoders.

3. **Indexing:** Saving embeddings into a fast-lookup Vector Store (e.g., Chroma, FAISS).
4. **Retrieval:** Comparing user queries against the index using vector similarity matching (Cosine similarity or Euclidean distance).
5. **Generation:** Injecting the retrieved context chunks directly into the LLM system prompt so it can answer with high precision and zero hallucinations.

## 2.4. Gradio UI

**Gradio** is an open-source Python library used to rapidly construct beautiful, customizable web demos for machine learning models. Using its native ChatInterface or lower-level blocks, Gradio handles websocket connections, session state tracking, markdown rendering, and audio/text stream management without requiring any frontend JavaScript boilerplate.

## 3. HANDS-ON GUIDES

### Task 1: Setup Development Environment

#### 1.1. Create Project Structure

Establish a clean project workspace matching the architecture below:

```
langgraph-chatbot/
├── data/
│   └── company_policies.txt    # Custom private knowledge data
├── app/
│   ├── __init__.py
│   ├── config.py              # API keys & environment config
│   ├── database.py            # Vector database initialization
│   ├── graph.py               # LangGraph architecture definition
│   └── ui.py                  # Gradio interface definition
├── main.py                    # Project entrypoint
├── requirements.txt           # Dependency file
├── tests/
│   └── test_routing.py         # Unit tests for routing rules
└── README.md                  # Documentation
```

#### 1.2. Setup Virtual Environment & Install Dependencies

Create a clean virtual environment and install the required core packages:

```
python -m venv venv
source venv/bin/activate # On Windows use: venv\Scripts\activate
```

```
pip install --upgrade pip
pip install langgraph langchain langchain-community langchain-openai chromadb gradio pytest
python-dotenv
```

## Task 2: Implement Knowledge Base & RAG Tooling

### 2.1. Ingest Private Knowledge Documents

Create a private knowledge file `data/company_policies.txt` containing company-specific policies, such as:

- Company travel allowance guidelines (\$150 per day for meals).
- Remote work eligibility policy (minimum 6 months tenure, hybrid schedule of 2 days onsite).
- Equipment policy (standard issue is an M3 MacBook Pro, refreshed every 3 years).

### 2.2. Implement Database Loader

In `app/database.py`, write a utility class to:

- Read `data/company_policies.txt`.
- Split the document into chunks (e.g., `RecursiveCharacterTextSplitter` with chunk size of 500, overlap of 50).
- Initialize a local, persistent vector store (e.g., Chroma or FAISS) using an embedding model (e.g., `OpenAIEmbeddings` or `HuggingFace embeddings`).
- Expose a retrieve function or standard vector store retriever.

## Task 3: Build the LangGraph State & Router Node

### 3.1. Define Global Graph State

Define what data travels through the system. You will declare a `TypedDict` class containing the user query, chat history, retrieved document context chunks, routing decision, and final generated output response.

### 3.2. Define the Routing Logic

Implement a routing function (node or edge conditional check) that analyzes the current state's messages.

You must implement a **Semantic Router**. It must call a structured LLM output (using schema definitions or JSON parsing) to evaluate the query:

- If the user's prompt inquires about company policies, travel, laptops, internal guidelines, or specific corporate operational processes, classify as "rag".
- If the user's prompt is a greetings, general knowledge request, coding help, mathematical problem, or casual conversation, classify as "general".

## Task 4: Construct the Specialist Node Agents

### 4.1. General Chatbot Agent Node

- Accepts the current graph State.
- Constructs a system prompt instructing the model to behave as a friendly, helpful general corporate assistant.
- Calls the LLM passing the user query and the chat history.
- Writes the resulting answer back into the State and completes.

### 4.2. RAG Agent Node

- Accepts the current graph State.
- Queries the retriever implemented in `app/database.py` with the user's question.
- Extracts the retrieved context chunks.

- Constructs a highly constrained prompt: **"Answer the user query ONLY using the provided retrieved context. If you do not know the answer based on the context, state that you do not have that information."**
- Calls the LLM and writes the final accurate response (and optionally source context) into the State.

### Task 5: Assemble the Compiled LangGraph Workflow

In app/graph.py, combine all elements:

1. Initialize a StateGraph object.
2. Add the nodes: general\_agent\_node, rag\_agent\_node, and (optional) routing\_node.
3. Define the entry point.
4. Define a **Conditional Edge** flowing from the initial routing decision to either the general\_agent\_node or the rag\_agent\_node.
5. Establish a transition from both agent nodes to the ending sequence (END).
6. Compile the graph with a memory saver (optional, for persistent memory tracking).

### Task 6: Build the Gradio UI

In app/ui.py, construct a unified chat interface:

- a) Set up a Gradio Blocks structure.
- b) Create a standard layout containing a gr.Chatbot display component, a gr.Textbox input component, and action buttons.
- c) Write a response generation function:
  - a. Takes the user query and previous conversational history list.
  - b. Formats the data into the LangGraph state format.
  - c. Invokes the compiled graph: graph.invoke({"messages": [HumanMessage(content=query)]}).
  - d. Extracts the final node outputs.
  - e. Displays the agent response along with secondary metadata indicating **which route was taken** (e.g., "[Routed via General Agent]" or "[Routed via RAG Agent]") so the user knows exactly how the graph executed behind the scenes.

## 4. DELIVERABLES & GRADING

### 4.1. Deliverables

Your team must submit a GitHub repository link containing:

- a) **Fully Functional Multi-Agent Graph:** Code base loaded and running dynamically using LangGraph.
- b) **Private Data File:** A customized corporate information source in data/company\_policies.txt.
- c) **Active Semantic Router:** Demonstrable LLM-based query-intent classification.
- d) **Unit Tests:** Tests using pytest verifying that the router correctly directs typical inputs (e.g. greeting prompts must map to "general", specific accommodation policy queries must map to "rag").
- e) **Interactive Web UI:** Executable Gradio server in main.py allowing interactive testing.
- f) **Comprehensive Documentation:** A README.md containing architectural explanations, setup instructions, and sample query logs demonstrating state changes.

### 4.2. Grading Rubric

| Criteria                                          | Weight     | Detailed Description                                                                                                                                                                                                                                                |
|---------------------------------------------------|------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>System Architecture &amp; LangGraph Design</b> | <b>30%</b> | StateGraph initialized properly (10%). States, nodes, and edges correctly connected with memory checkpoints (10%). Correct handling of chat history arrays within the LangGraph State representation (10%).                                                         |
| <b>Intent Router Accuracy</b>                     | <b>20%</b> | Semantic classifier routes corporate policies queries to the RAG Agent with $\geq$ accuracy (10%). Dynamic routing edges execute correctly in real-time without infinite loops or invalid node transfers (10%).                                                     |
| <b>RAG Performance &amp; Integration</b>          | <b>20%</b> | Document chunking, indexing, and vector store configuration (10%). Context-conditioned system prompting preventing hallucinations on unknown policies (10%).                                                                                                        |
| <b>Gradio User Interface</b>                      | <b>15%</b> | Responsive multi-message chat panel (5%). Visual indicator displaying the execution route metadata (General vs. RAG) on every message turn (5%). Proper application responsiveness and error notifications in UI (5%).                                              |
| <b>Testing &amp; Documentation</b>                | <b>15%</b> | At least 6 functional automated tests (pytest) covering route classification limits (5%). Clear README with run commands, architecture design, and explicit environment variables config (5%). Clean, PEP-8 compliant code structure with complete docstrings (5%). |

### 4.3. Submission Rules

- **Deadline:** 1 week after the scheduled lab session.
- **Format:** Submissions must be delivered as a single packaged .zip file containing clean source code, document data, and a valid GitHub repository URL. Do not include your .env or venv folders inside the submitted archive.

lows